

COMP90024 - Assignment 2 Report

Houston Karnen 1254942, Brian King 1537997, Klarissa
Jutivannadevi 1571487, Jaden Ling 1263521, Lachlan Hugo 696147

Abstract

This report provides an analysis of traffic data and its relationship with air quality metrics. Real-time data was collected from the Bureau of Meteorology (BoM), Environmental Protection Authority (EPA), and traffic monitoring services. The data collection process used Kubernetes on the Melbourne Research Cloud (MRC) to ensure scalability and reliability.

Fission harvesters were implemented to fetch, transform, and load data into an ElasticSearch database. These harvesters operate continuously, supplying updated data for analysis. The collected data was processed and visualised using Jupyter notebooks, allowing for detailed examination of traffic patterns and air quality trends.

Visualisations highlight the changes in traffic flow and its impact on air quality across different freeway segments. Key metrics such as actual travel time, PM10, PM2.5, and other pollutants were analysed to identify significant patterns and correlations. This approach provides insights into the connection between traffic congestion and environmental health, supporting better decision-making for urban planning and public health.

Overall, the use of a scalable, automated data pipeline on MRC, combined with analytics in Jupyter, shows an effective method for real-time environmental monitoring and analysis. This study emphasises the importance of using cloud-based infrastructure and serverless computing to manage large-scale data collection and processing tasks efficiently.

Table of Contents

Introduction.....	3
Methods.....	4
System Architecture.....	4
Kubernetes.....	5
Fission Harvesters.....	6
Fission API.....	7
Elasticsearch.....	8
Jupyter Notebook.....	9
Implementation.....	9
Data Sources.....	10
VicRoads Traffic Data.....	10
SUDO Vehicle Registrations.....	11
EPA Air Quality Data.....	12
BOM Weather Data.....	12
SA2 Spatial Boundaries.....	12
Analytics and Scenarios.....	13
Fault Tolerance and Scalability.....	21
Errors.....	21
Scalability.....	22
Results.....	23
Discussion.....	24
Difficulties.....	25
Difficulties in Kubernetes.....	25
Difficulties in Fission.....	25
Difficulties in Elasticsearch.....	25
Difficulties with cloud technologies (MRC).....	26
Conclusion.....	26
Demonstration Link, Repository Link, Table of Contributions.....	27

Introduction

The interplay between traffic congestion and air quality is a critical issue in urban environments, directly impacting public health and quality of life. Understanding the relationship between these factors requires a robust and efficient data collection and analysis framework. This report presents an integrated approach to monitor and analyse traffic data alongside air quality metrics using advanced data collection and processing technologies.

Data for this study was sourced from the Bureau of Meteorology (BoM), Environmental Protection Authority (EPA), and various traffic monitoring services. The data collection process was managed using Kubernetes deployed on the Melbourne Research Cloud (MRC). Kubernetes provides a scalable and reliable platform to handle the dynamic nature of data from multiple sources.

The core components of the data pipeline are fission harvesters. These are serverless functions designed to fetch, transform, and load data into an ElasticSearch database. The continuous operation of these harvesters ensures that the data is always up-to-date, enabling real-time analysis. The use of ElasticSearch as the data storage solution allows for efficient indexing and retrieval of large datasets, which is crucial for performing timely and accurate analyses.

Jupyter Notebooks were used for data processing and visualisation. This platform offers a flexible and interactive environment for analysing the data and creating detailed visualisations. By examining metrics such as actual travel time and pollutant levels (e.g., PM10, PM2.5), the study aims to uncover patterns and correlations that can inform urban planning and public health strategies.

The visualisations generated in this study provide insights into how traffic conditions influence air quality across different freeway segments. By identifying these patterns, stakeholders can develop more effective interventions to manage traffic flow and improve air quality. This integrated approach demonstrates the importance of using advanced cloud-based infrastructure and serverless computing to address complex environmental challenges.

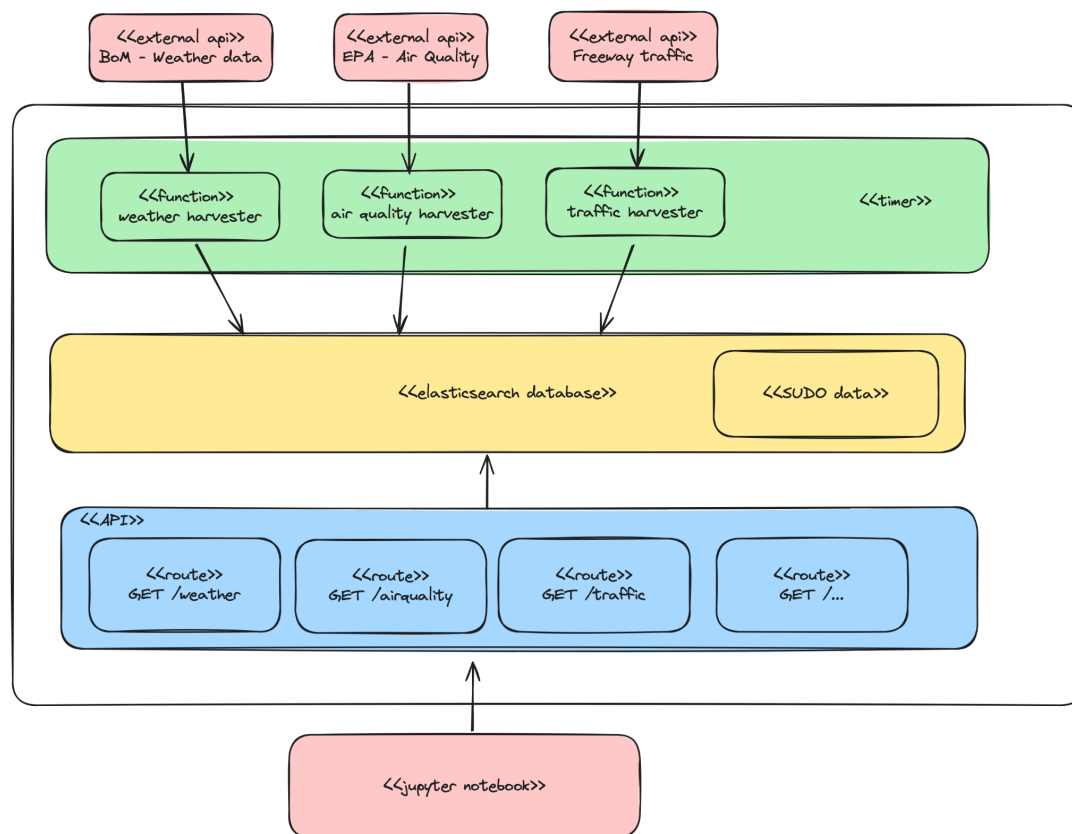
The following sections of this report will detail the methodologies used in data collection, processing, and analysis, present the findings from the visualisations, and discuss the implications of these findings for urban planning and public health.

Methods

System Architecture

This section describes the technologies and software used to build our data analysis application. Our goal was to create a robust and scalable system capable of ingesting, processing, and supporting the analysis of large datasets from various sources.

The system architecture we designed for this project is built on the services provided by the Melbourne Research Cloud (MRC), predominantly through the deployment of various microservices on a Kubernetes cluster. This architecture allowed us to develop different components of the application which would support the ingestion, storage, and presentation of various data sources, culminating in a Jupyter Notebook serving as the frontend of our system.



Kubernetes

A Kubernetes cluster forms the basis of our infrastructure for this project. Kubernetes is a container orchestration engine which supports the automated deployment, scaling and management of containerised applications. It ensures the desired system state, as specified in the configuration, is achieved even in the presence of failures. For example, if containers or services become unresponsive, Kubernetes will automatically restart them. If an entire node in the cluster fails, Kubernetes will reorganise the containers across the functioning nodes to maintain service availability.

In the context of building our application, Kubernetes allows us to focus on building stable, self-contained services. We deploy these services to the cluster using configuration files, leaving Kubernetes to handle the specifics of deployment based on resource availability and utilisation across the cluster.

We provisioned the Kubernetes cluster using the OpenStack Magnum service, defining a cluster with four nodes, including one master node. On this cluster we deployed three applications to support our project: Elasticsearch, Kibana, and Fission.

- Elasticsearch was configured with 2 nodes or replicas, each provisioned with 100GiB of persistent storage. When configured this way, Kubernetes ensures that Elasticsearch can handle large data volumes and remain operational even during node failures.
- Kibana was installed as a GUI frontend, supporting us to visualise and manage various aspects of the Elasticsearch deployment.
- Fission was installed as a serverless framework, allowing us to deploy self-contained functions to perform various tasks, including data harvesting from external services and exposing Elasticsearch data via a RESTful API. Kubernetes manages the execution of these functions, providing a resilient platform on which to deploy our application functionality.

By leveraging Kubernetes as the foundation of our application deployment, we enable ourselves to scale for potential future demand and maintain high availability. In addition, it allows us to focus on developing the core services of our project, leaving it to Kubernetes to handle the operational complexities associated with hosting a microservice-based application.

Fission Harvesters

At the core of the data collection process are the Fission serverless functions, called harvesters. These functions fetch, transform, and load data from our key data sources—the Bureau of Meteorology (BoM), Environmental Protection Agency (EPA), and VicRoads—into the Elasticsearch database. Fission’s serverless architecture enables these functions to run on-demand, ensuring efficient use of resources.

In the case of the BoM and EPA data, both data sources only refresh hourly. Therefore, the harvesters for these sources are configured to run once every hour. The data available from the VicRoads freeway traffic API updates far more frequently, with new traffic data available every 30 seconds. This provides granular access to congestion and travel times across the freeway network. To efficiently handle this data, the harvester is configured with a time trigger which collects new data every minute. Given the near real-time update of this data source, we might consider this to be pseudo-streaming into our database.

Our distributed architecture incorporating Elasticsearch is well equipped to handle the rapid ingestion of data. The use of Fission serverless functions allows us to efficiently spin up resources as they are required to process incoming data. Both elements make this an inherently scalable solution, with the Kubernetes cluster supporting scaling out to additional nodes and increasing compute capacity if more data sources are added or if the data velocity increases.

Fission API

In addition to data harvesting, Fission functions facilitate interaction between the Jupyter Notebook front-end and Elasticsearch database through data queries for analysis and visualisation. These functions are exposed as RESTful APIs, creating an abstraction layer between the external users of the data and the implementation logic in the backend. Exposing the data through the RESTful API allows end users to discover all resources available to them, that is, the list of all available weather stations etc. (through the GET /weather-stations endpoint), and the aggregated weather data available for each station (GET /weather-stations/{station_id}).

Following REST best practices, we have preferred paths over query strings, employing the latter only where appropriate to modify request parameters. The implementation of the queries is performed in the Python functions, using the Elasticsearch library and Elastic DSL to perform searches and aggregation within data stored on the Elasticsearch cluster. This means that queries to the API are highly performant as they leverage Elasticsearch's optimised search capabilities.

Exposing the data through the API means that the system maintains flexibility to change implementation details as needs evolve. This could include adding new indexes or ways of storing the original data, or switching to an entirely new database solution. Abstracting the data

storage from the end user allows for changes without impacting the contract provided to end users through the API. Additionally, if the format of one of the data sources were to change upstream, this could be handled and the data normalised before it reaches the end user, ensuring service stability.

Function	Method	Route	Parameters	Details
weather-stations	GET	/weather-stations	-	Gets the list of all weather stations, their coordinates, and names
weather-stations-observations	GET	/weather-stations/{station_id}	Year, Month, Day, Hour	Returns aggregated temperature and wind data for a weather station
air-quality-stations	GET	/air-quality-stations	-	Gets the list of all air quality stations, their coordinates, and names
air-quality-stations-observations	GET	/air-quality-stations/{station_id}	Year, Month, Day, Hour	Returns aggregated air quality data for an air quality station
traffic-freeway	GET	/traffic-freeway	-	Gets the list of all freeways
traffic-observations	GET	/traffic-freeway/{freeway_name}	Year, Month, Day, Hour	Returns aggregated congestion and freeway segment data for a freeway
sudo-vehicle	GET	/sudo-vehicle	-	Returns aggregated data on total dwellings and dwellings with at least one motor vehicle per SA2 area

Table 1: List of all APIs

Elasticsearch

Elasticsearch serves as the database for this study, storing data in the form of JSON documents from the Fission harvesters in three indexes, along with data manually downloaded from SUDO. Given the scope of this project, Elasticsearch is an excellent option for a database backend, giving us the ability to easily store semi-structured data coming from many different sources and providing concurrent read access across multiple nodes. Since the primary operations for our database application are insert and read access, (updates are not generally required for this project's purposes), Elasticsearch is well-suited to meet these needs.

Additionally, it is excellent at retrieving time-based data, which is crucial in our context as we are primarily storing time-stamped data, at intervals ranging from hourly to minute-level and need the ability to filter and aggregate across various timespans. By mapping data types in our documents and correctly specifying date-time formats, we leverage Elasticsearch's highly performant search capabilities. Through the utilisation of the Query DSL in our Fission API, we are able to offload the searching and aggregation to Elasticsearch, enabling our API to respond to requests quickly and efficiently. This capability supports real-time analysis in the Jupyter Notebook frontend, allowing users to interact with and visualise data dynamically.

Jupyter Notebook

Finally, a Jupyter Notebook serves as the frontend for this application, enabling powerful analysis and visualisation to explore scenarios inspired by our data. For system abstraction and performance reasons, the notebook retrieves the data stored in the database by querying a RESTful API. This approach allows users to analyse and display information needed to investigate relationships between air quality, weather, traffic, and car ownership. By leveraging Jupyter Notebooks in this way, we provide a simple but powerful interface for exploring complex datasets.

Implementation

To implement the various components of our system, our team primarily used Python as the scripting language. Python's extensive libraries and frameworks made it an ideal choice for developing our data processing and analysis functions.

As described above, we used Elasticsearch Query DSL to ensure efficient resource utilisation and performant data queries. Following best practices for Kubernetes, we specified our infrastructure as code. All our Fission deployments were defined through YAML configuration files and applied to the cluster. This approach ensured consistent deployments and made it easier for team members to make updates. Additionally, tracking all changes through version control provided a clear history of modifications and facilitated collaboration.

Data Sources

Here we provide a summary of the various data sets that we sourced to undertake our analysis. We used two primary types of data:

1. static datasets retrieved through SUDO and the Australian Bureau of Statistics (ABS)
2. real-time data sourced from the web services of the following public data providers:
 - a. The Bureau of Meteorology (BoM),
 - b. The Environmental Protection Agency (EPA)
 - c. VicRoads

VicRoads Traffic Data

The primary source of traffic data is the VicRoads API, which provides real-time access to travel conditions across all freeways in the Melbourne metropolitan area. This data is fetched through a Fission function, *traffic-harvester* and configured with a time interval trigger to retrieve the latest data from the API every minute. Across the duration of our project we had plenty of storage space in our cluster to accommodate this retrieval frequency. We address alternative approaches which might have better scalability for a long running project later in the report.

The retrieved data includes various metrics such as average vehicle speeds, average travel time, freeway coordinates, and congestion levels. To ensure storage efficiency (and focus the analysis) the data was preprocessed to retain only relevant information. The following data fields are stored in ElasticSearch, in the *traffic-index*:

Field Name	Description
obs_id	A unique identifier for each data point, constructed by concatenating the freeway ID with the timestamp, e.g., ‘freewayID—YYYY–MM–DDTHH:MM:SS’.
freewayName	The name of the freeway.
segmentName	The name of the segment of the freeway.
publishedTime	The exact time the data was published, concatenated into obs_id as well.
condition	The traffic condition, e.g., heavy, light, medium.
actualTravelTime	The actual time taken to travel the segment during the observed period.
averageSpeed	The average speed of vehicles on the segment during the observed period.
congestionIndex	A numerical value representing the level of congestion.

Table 2: *traffic-index* data fields and description

SUDO Vehicle Registrations

Because the traffic data was only available for Victorian freeways and therefore had limited geographic spread, we sought alternative ways that we might estimate vehicle density to supplement our analysis. To this end, we incorporated the data available on SUDO of vehicle registrations at the SA2 geographical level as a proxy for traffic density in areas that were not covered by the freeway data. This supported our analysis to investigate whether regions with higher numbers of registered vehicles have poorer air quality, which could be inferred to be influenced by the higher emission levels caused by the increased number of cars.

EPA Air Quality Data

We sourced air quality data for Victoria from the Victorian Environmental Protection Agency API. The API provides access to air quality data from many sites, with different sites offering different air quality metrics. PM2.5 was the most consistently available metric across available sites, so we decided to use this as the primary air quality metric for our analysis.

The data available through the EPA API is updated hourly, so a fission function, ***EPA-harvester***, *was* configured to extract all data available from the API hourly and insert it into the ***air_quality_data*** index in the Elasticsearch database. The following fields were stored to support future analysis

BOM Weather Data

General weather data for Victoria was sourced from the Bureau of Meteorology weather service, which provides hourly weather observation data for many weather stations across Victoria. The observations include temperature, humidity, wind speed, among others, making them a comprehensive dataset for our environmental analysis. We configured a fission function, ***BOM-harvester***, to retrieve the data hourly and insert it into the ***weather_data*** index in our Elasticsearch database.

Digital boundary files were sourced from the Australian Bureau of Statistics (ABS) and used for spatial analysis in conjunction with the SA2 level vehicle registration data obtained from SUDO.

Our analytics capture several scenarios revolving the data we capture, these include traffic, air quality, and weather. The scenarios vary - in which some scenarios are dependent on only one dataset, while some other scenarios capture the relationships between 2 or more datasets.

A map of Melbourne, Australia, displaying various data points. The map includes major roads (M1, M3, M31, M79, M8, M420), parks (Brisbane Ranges National Park, Future Western Grasslands Nature Conservation Reserve), and water bodies (Port Phillip). A legend in the bottom left corner identifies three types of data points: Traffic Data (red pin), Air Quality Stations (blue pin), and Weather Stations (green pin). The map shows a high density of Traffic Data points in the central urban area, several Air Quality Stations along the coast and in the center, and Weather Stations scattered across the region, including in the western and southern parts.

The first two analysis focuses on air quality and wind speed distribution based on different locations, using the map to explain how location affects results.

Air Quality on Different Locations

Air quality is compared based on PM2.5 level (fine particles that are significant to air pollutants). This compares PM2.5 levels in different locations. Locations nearer to CBD are expected to have higher PM2.5 level due to high densities of people in that area.

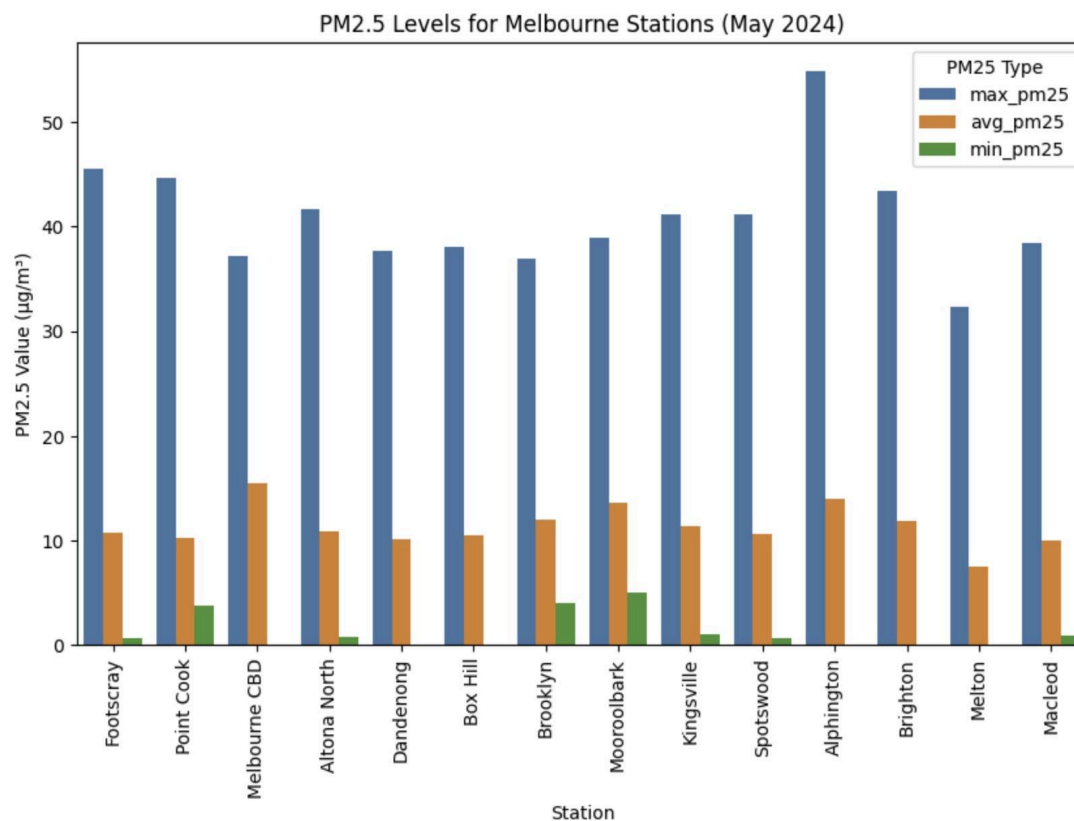


Figure 2: Comparison of Air Quality (PM2.5 Value) on Various Melbourne Air Quality Stations

From Figure 2, it seems like Alphington has the highest PM2.5 level; while several locations, including Alphington, also have 0 PM2.5 value recorded as the minimum. And the rest is pretty much the same for every location, where they had a shorter gap between the average and lowest PM2.5 level than the maximum PM2.5 level.

The bar chart does not entirely support our initial hypothesis. Previously, since there is not much data known other than different locations and PM2.5 value, we assumed that location nearest to CBD would have the highest PM 2.5 level, as we know that people contribute to the increased

level of PM2.5 and CBD is the most densely populated location in Melbourne. The reason why this is not completely true is because the graph clearly shows that Alphington has the maximum level of PM2.5 value, which is more than 10 levels higher than the maximum PM2.5 level in CBD. However, we can also see that Melbourne CBD has the highest average PM2.5 level, which could support our hypothesis. This means that the spiked up PM2.5 level in Alphington on that day might be caused by some external factors we are not aware of.

Wind Speed Distribution on Different Locations

The analysis compares the strength of wind in different locations. Locations surrounded by more oceans are expected to have higher strength of wind.

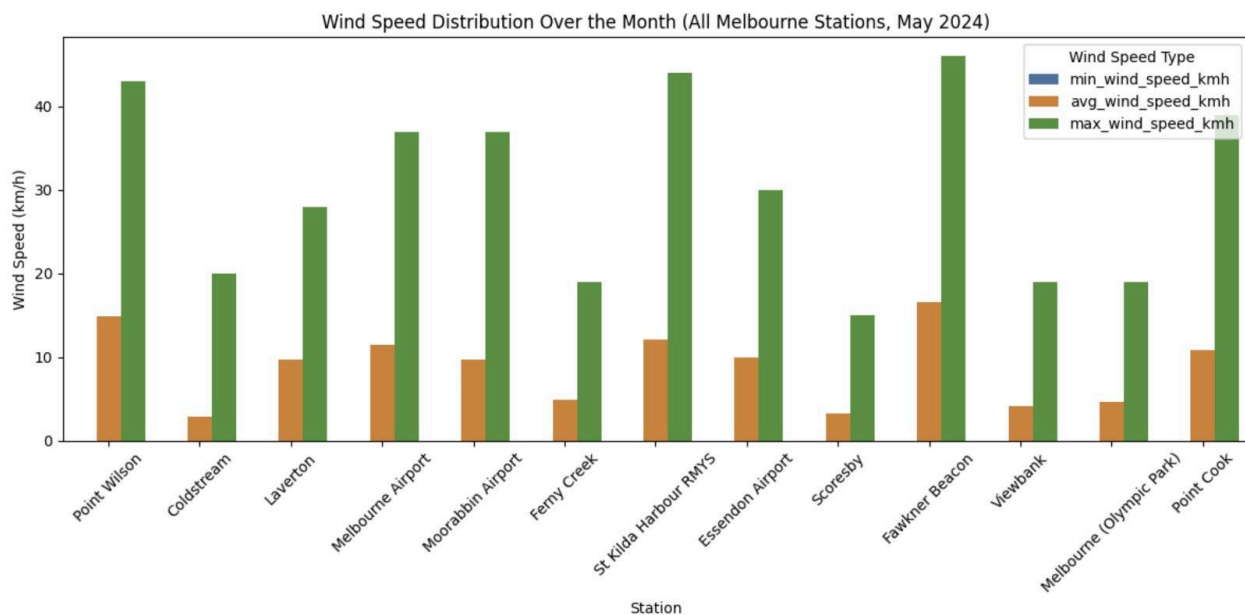


Figure 3: Comparison of Wind Speed on Various Melbourne Weather Stations

The graph above compares the wind speed in several locations in Melbourne. The data shows that Fawkner Beacon has the highest maximum wind speed and average wind speed, while Scoresby has the lowest maximum wind speed, and slightly higher average wind speed compared to Coldstream. However, it is observed that there is 0 minimum wind speed for every location, which is justifiable since there are some days where there is no wind.

The result agrees with our initial hypothesis that locations nearer to the oceans will have higher wind speed. Based on the geographical area, the 4 locations with highest maximum wind speed are located near the sea. To further support our hypothesis, Fawkner Beacon has the highest wind speed and is the most isolated location, only being surrounded by sea. Locations like Coldstream and Scoresby are not only far from the ocean, but are also densely populated with buildings, hence why their maximum wind speed is so low compared to the ones near the ocean.

The analysis below focuses on freeway data, providing insights on how different metrics can affect the air quality in a certain area.

Congestion Index Across Freeways

This compares the peak congestion levels across the freeways. Higher congestion levels are expected to result in higher emissions, which can affect air quality in the area.

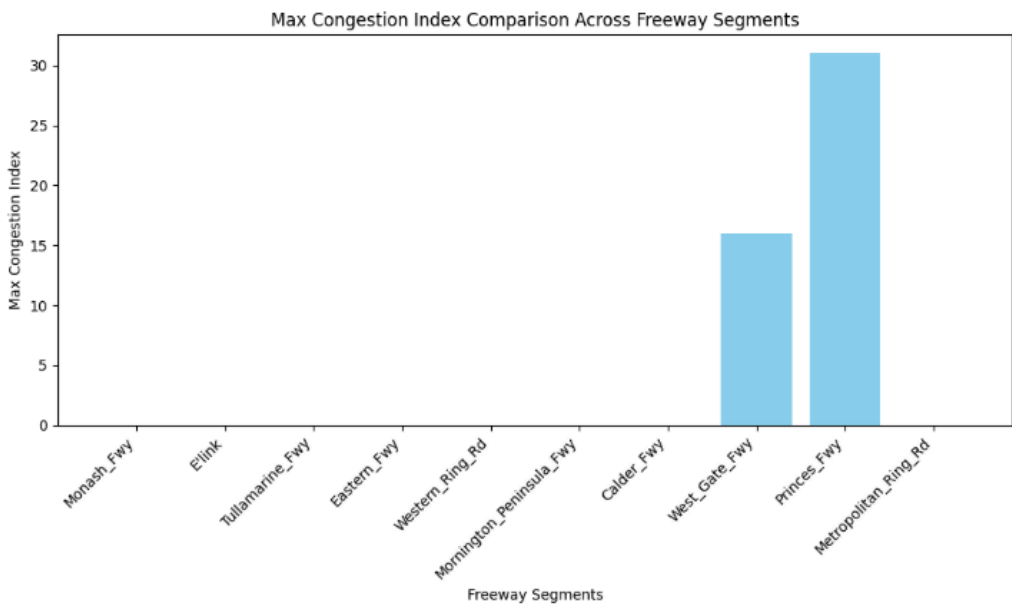


Figure 4: Congestion Index Across Freeway Segments in Melbourne

The data shows that the majority of freeway segments experience zero or minimal congestion. However, West Gate Freeway and Princes Freeway are notable exceptions, both of which have significantly higher congestion indices.

Travel Time Across Freeways

Another analysis is to compare the actual travel time across the freeways. We hypothesize that longer travel times are expected to cause worse air quality in the area, due to more emissions.

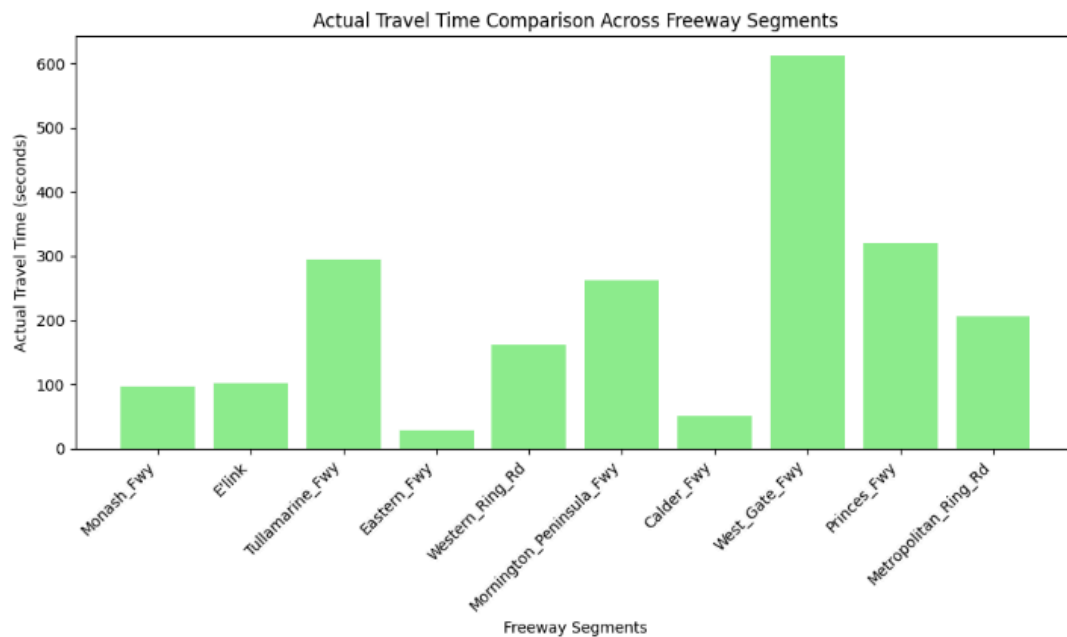


Figure 5: Actual Travel Time Across Freeway Segments in Melbourne

Figure 5 above reveals that travel times on most freeway segments average around 100 seconds. Variations can be attributed to actual lengths of the freeways. However, West Gate Freeway and Princes Freeway both have very long travel times. Eastern Freeway and Calder Freeway, though, have the shortest travel time which is less than 100 seconds.

Congestion Index vs. Travel Time

This analysis explores the relationship between peak congestion levels and the actual travel time on various freeway segments. We hypothesize that higher congestion levels significantly increase travel times, leading to higher emissions and worse air quality.

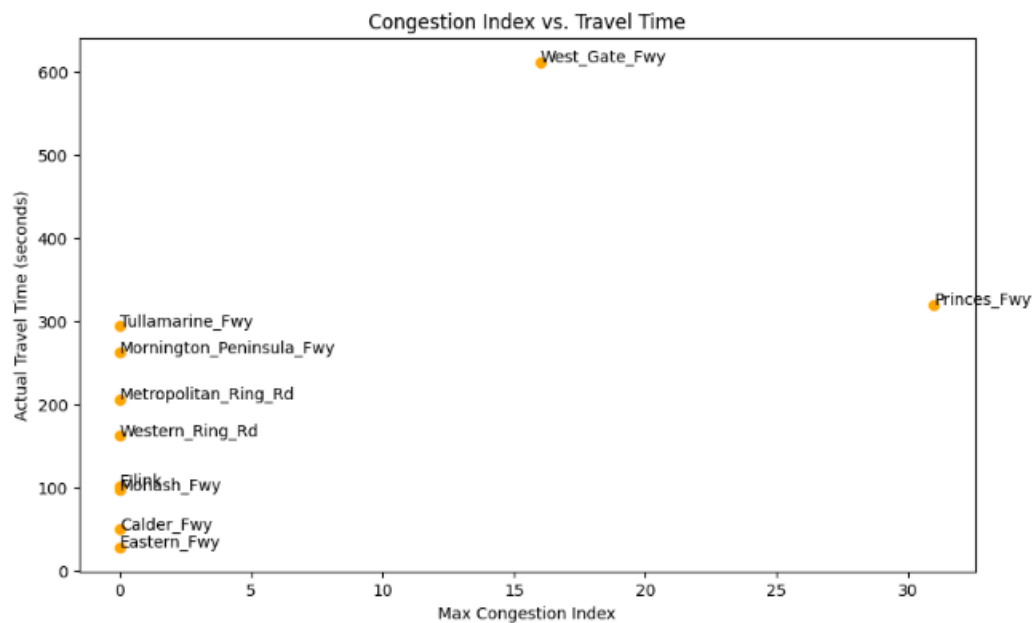


Figure 6: Scatter plot of Actual Travel Time and Maximum Congestion Index on Different Locations in Melbourne

Figure 6 combines the data obtained from figures 4 and 5 above, showing how congestion index potentially correlates with higher travel times. As can be seen, higher congestion freeways have higher travel times. However, since the data is not well spreaded - with most freeways having

minimal congestion, it is hard to draw meaningful conclusions here. There could be other factors affecting the results.

Average PM2.5 level vs. Actual Travel Time

This analysis showcases the relationship between average PM 2.5 level and actual travel time. Our hypothesis is that, the higher the actual travel time, the higher PM2.5 level be. This is because higher travel time means the longer vehicle engines are on and create PM2.5 particles.

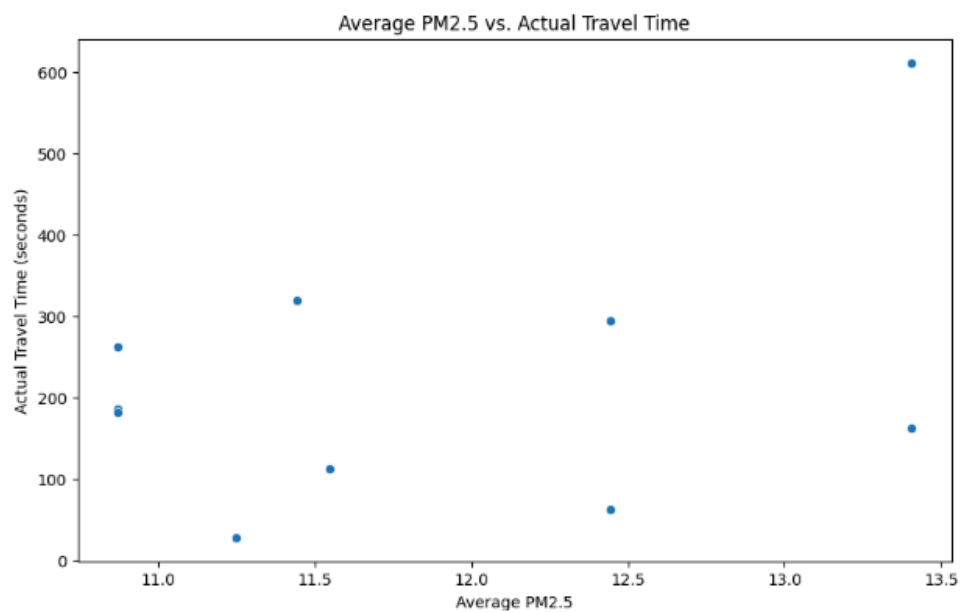


Figure 7: Average PM2.5 level and Actual Travel Time

The scatterplot shows a random distribution observed from the data. For instance, there are two freeways having an average PM2.5 level of 13.5 but have a time difference of more than 400 seconds. There is no clear correlation whether the actual travel time affects the average PM2.5

level. Hence, our hypothesis on how average PM2.5 and actual travel time are positively correlated are not completely true.

Correlation Matrix

The heatmap of the correlation matrix provides a comprehensive view of the linear relationships between various numerical features in the dataset. Each cell in the matrix represents the correlation coefficient between two features, with a colour gradient indicating the strength and direction of the correlation. This visualisation is essential for identifying strong positive or negative correlations, such as how closely related maximum congestion index might be with actual travel time, or how weather variables might influence air quality metrics. The heatmap aids in feature selection for predictive modelling and understanding underlying data patterns. When choosing our features, the key thing to consider is the multicollinearity between variables. For example, total dwellings and number of vehicles per dwelling have a correlation of 1, indicating that they are perfectly correlated with each other and only one of these variables will suffice for the final model. Ultimately, the features chosen were max congestion index, actual travel time, max wind speed, average temperature, and number of motor vehicles per dwelling

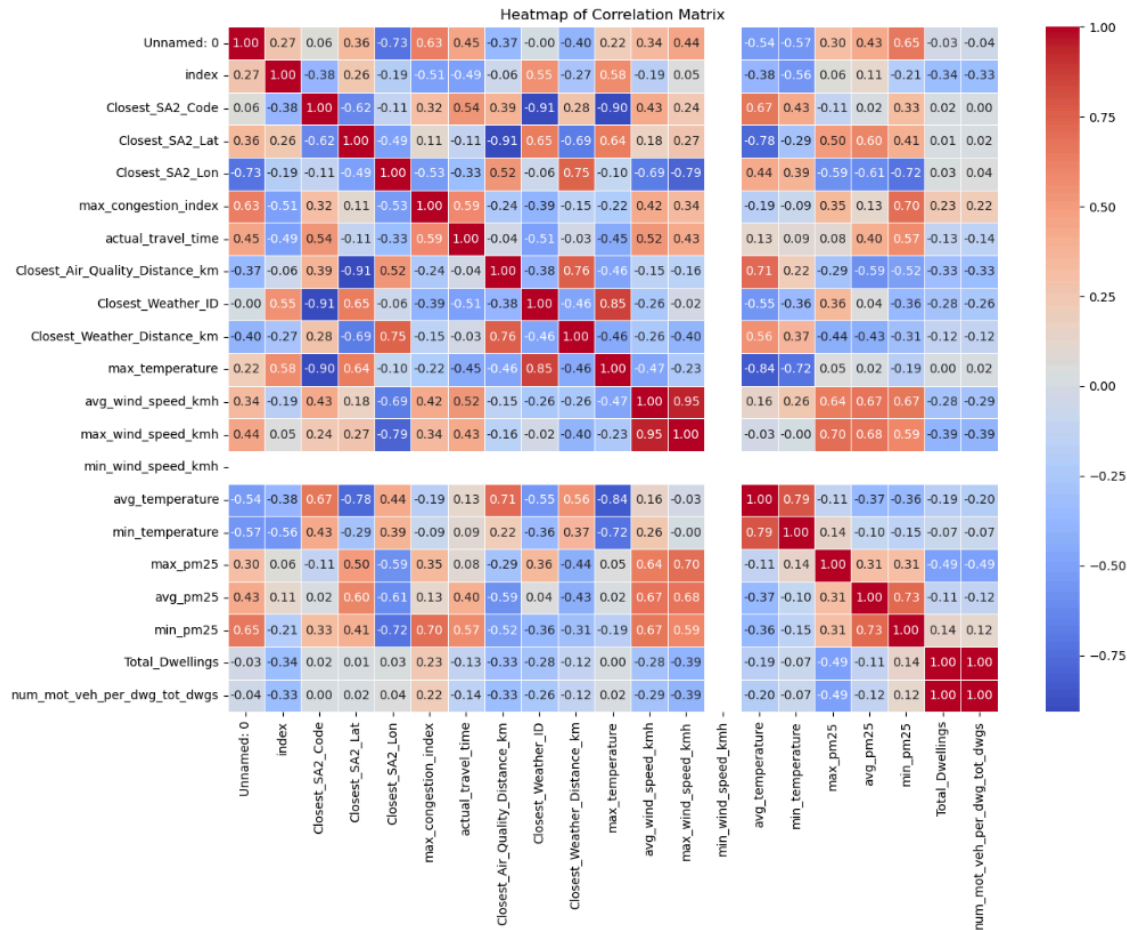


Figure 8: Heatmap on All Variables and Each of the Correlation

Fault Tolerance and Scalability

Errors

Robust error handling was vital to the success of this project given the number of data sources and the frequency of external data retrieval. In particular, errors in connecting to external data sources needed to be handled by the harvesters to ensure that we were alerted to connection issues or changes in the format of the retrieved documents.

HTTP errors were a particular concern for the EPA API which has a strict limit of 6 requests per second. This required adding delays between requests and carefully checking response status codes to wait and retry if the limit was exceeded.

The *traffic-harvester* interacts with the VicRoads API, which updates every 30 seconds. To account for any potential delays or inconsistencies in the AP, the harvester only fetches data every minute. This provides a buffer which ensures the integrity of the data collected, reducing the risk of capturing incomplete or duplicate entries. Additionally, the harvester only accepts successful responses (HTTP status code 200), ignoring all others to maintain data integrity. This approach works as there are no rate limits on the API calls.

The weather data is particularly vulnerable to changes on the server end since the data isn't retrieved through an official API and lacks the data stability guarantees of a dedicated API. Additionally the latitude and longitude coordinates in the weather observation data were imprecise, requiring us to source a fixed-width file from the BoM containing all station coordinates to join with each observation. This is a particularly fragile part of the harvester, so we added careful logging to notify us if the source file changes and cannot be joined, defaulting to the less precise coordinates provided in the original observation.

A potential point of failure in the system is the interaction between the harvesters and the database. The current implementation relies on each harvester transforming and loading data into the database. This means that if the Elasticsearch database were to experience a failure, the harvesters would fail at insertion, potentially resulting in data loss.

An alternative approach could be to use a message queue (e.g., Apache Kafka, as shown briefly in a workshop) between the harvesters and the database. This way, if the database is down, the data could be stored in the queue until the database is back online, at which point the database would be populated with the entries waiting in the queue.

Scalability

Elasticsearch is a highly scalable solution. We can envision expanding this project to include weather, traffic, and air quality data beyond Victoria to other states and territories. One consideration in this scaling could be where to store data for optimal end-user performance. For instance, a Victorian user might most frequently access Victorian data, whereas a Perth user might be most interested in Perth data. Ensuring the fastest access to relevant data could involve

distributing the database storage across various nodes throughout the country, sharding the database so that data is stored on nodes closest to where it is most likely to be accessed.

The frequency of data arriving from the VicRoads API at 60-second intervals could pose challenges for storage depending on the project's lifespan. Depending on analysis and reporting needs, it might be possible to leverage Elasticsearch lifecycle policies for more sophisticated long-term data management. For example, you might choose to introduce an index management system that keeps the most recent traffic data in a 'hot' state. As the data ages, it can be moved to a 'cooler' state where it might eventually be deleted or stored permanently in an aggregated form.

Additionally, implementing a more robust logging mechanism for tracking the performance of external data retrieval is crucial. As the number of external data sources increases, it becomes less viable to manually check that all the harvesters are working as expected. While the logging within the Fission functions is useful, enhancing it to provide more comprehensive tracking and alerting will be essential for maintaining system reliability and performance.

Results

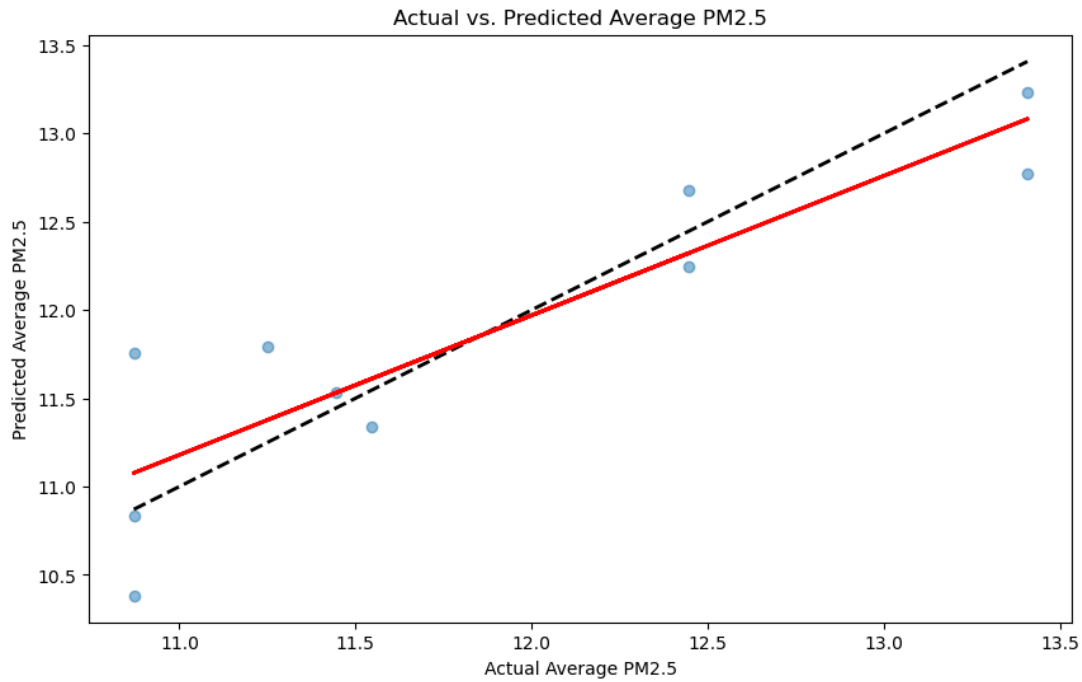


Figure 9: Linear Regression of Predicted and Actual Average PM2.5 level

Mean Squared Error (MSE): 0.1898

The MSE represents the average squared difference between the actual and predicted values, where a lower value is indicative of a better fit. In this case, a value of 0.1898 suggests that the model's predictions are relatively close to the actual values albeit there is still some error.

R-squared Score: 0.7903

The R-squared value shows the proportion of variance in the dependent variable that is predictable from the independent variables. A value of 0.7903 demonstrates that 79.3% of the variance in air quality can be explained by the features selected, which is evidence of a good fit.

Discussion

Since we wanted to find the relationship between PM2.5 level with wind speed and traffic, it is best to include variables from our data that relates to them. Hence, the parameters used to create the linear regression shown in Figure 9 are *max_congestion_index*, *actual_travel_time*, *max_temperature*, *avg_wind_speed_kmh*, *max_wind_speed_kmh*, *min_wind_speed_kmh*, *avg_temperature*, *min_temperature*, *Total_Dwellings*, *num_mot_veh_per_dwg_tot_dwgs*.

The linear regression analysis plots the actual vs. predicted average air quality (PM2.5) levels, aiming to predict air quality based on several predictors including congestion, travel time, temperature, wind speeds, and car registry data. Each point in the scatter plot represents a freeway segment, with the x-axis showing the actual air quality levels and the y-axis showing the predicted amount by the model. The red line represents the best linear fit, indicating how well the model predictions align with the actual values. While the black dashed line represents the line of perfect prediction ($y=x$). Ideally, points should cluster closely around the line, indicating a good predictive performance. Deviations from this line highlight prediction errors, helping in diagnosing model accuracy and areas needing improvement. The plot offers a visual assessment of the model's ability to predict air quality, which is crucial for validating the model and making informed decisions based on its predictions.

Based on the analysis of the plot and the performance metrics, we can infer that the model demonstrates a strong ability to predict air quality levels based on the selected features. The points in the scatter plot are close to the line of perfect prediction, and the MSE is relatively low. The high R-squared score (0.7903) indicates that the selected features (max congestion index, actual travel time, max wind speed, average temperature, and number of motor vehicles per dwelling) are indeed useful in predicting air quality. Therefore, it suggests that the features used are valuable for predicting air quality, and the model itself provides reasonably accurate predictions. This supports the usefulness of these features in predicting air quality, although further refinement of the model could enhance its predictive accuracy.

Difficulties

Difficulties in Kubernetes

One of the difficulties with Kubernetes is with scaling and resource management. It is hard to configure auto-scaling properly in the Kubernetes cluster due to provisioning. We are required to configure autoscale so that the cluster can handle varying workloads. However, if the resources are over-provisioned, it can lead to increased costs; while under-provisioning can result in performance issues.

Difficulties in Fission

Since Fission is a serverless framework, it often experiences cold start latency when functions are invoked after being inactive for a period of time. This results in slower response time for users interacting with the system during the initialisation process. Additionally, being a serverless framework, Fission has limitations on execution time and resource usage, which restricts its ability to handle large or complex tasks. To address this, we need to break down these tasks into smaller functions, thereby managing the system's complexity more effectively.

The main challenge in using Fission is due to its non-interactive environment. Unlike local development, where code can be tested iteratively, Fission requires functions to be fully deployed before they can be tested. Another issue arises from inconsistencies in Fission function and package deployment and execution. Occasionally, deploying the same function or package under identical conditions yields different results; it may fail at one time and then succeed the next without any changes. Similarly, API calls sometimes fail without apparent reason, possibly due to issues with the MRC.

Difficulties in Elasticsearch

The main difficulty encountered with Elasticsearch is when dealing with data consistency. Elasticsearch is eventually consistent, which means that during the transient period, data will be inconsistent. The inconsistencies directly impact the real-time analytics and search accuracy.

which leads to the result analysis being different. Also, Elasticsearch can be resource-intensive; hence, we need to make sure that our computer is running too many tasks at the moment, to ensure that Elasticsearch can perform optimally. Even if CPU has been managed carefully, Elasticsearch clusters can become challenging to scale and will perform slower as data volume grows. This results with search query being slower, and indexing taking longer time.

Difficulties with cloud technologies (MRC)

Cloud technologies require the use of virtual machines; an inefficient use of allocated virtual CPUs and storage often results in performance bottlenecks, making the system experience slow processing time and delays in data retrieval and analysis. Accessing MRC is especially harder since there are many students accessing the MRC at the same time, leading to processing times being exponentially slower. On top of this, we also encountered issues where data is not persistent. The dynamic nature of cloud environments often leads to instances being terminated or replaced; this could potentially cause data loss if persistent storage is not managed properly. It's possible for critical data to be lost, leading to disruptions in data processing and analysis workflows. We also experienced some difficulties when the Fission calls slowed down due to the MRC; we just learnt that the issue lies with Cinder, which is beyond our control.

Conclusion

This study provided comprehensive insights into the correlations between traffic congestion, air quality, and weather data across Melbourne. Utilizing the Melbourne Research Cloud and serverless computing architecture facilitated efficient handling and processing of large-scale data sets. The analysis revealed significant correlations between high traffic volumes on certain freeways, notably the West Gate and Princes Freeways, and poor air quality indicated by PM2.5 levels. A linear regression model quantified these relationships, demonstrating through its performance that the selected features effectively captured the underlying patterns.. This proved our initial findings, concluding that there is a significant correlation between air quality and traffic.

To further enhance this study, several improvements are proposed:

- **Additional Data Sources:** Incorporating data on vehicle emissions, vehicle types, real-time public transportation usage, and population density could provide deeper insights into the factors most affecting air quality.
- **Advanced Machine Learning Models:** The current study uses basic linear regression models for predictive analysis. Future studies could explore more complex models such as neural networks or ensemble methods that may capture relationships in data more effectively.
- **Improving Serverless Architecture:** Minimize cold start latencies and optimize data flow by removing unnecessary features for data processing and analysis.
- **Additional Scenarios:** Conducting long-term studies over various timescales - such as times of day, seasons, years - would aid in understanding long-term traffic and air quality patterns.
- **Interactive Frontend:** Although Jupyter Notebooks serve well for data analysis, developing a dedicated interactive web application would provide a more user-friendly interface for clients to explore data and results independently.

Demonstration Link, Repository Link & Table of Contributions

[GitLab Repository](#)

Youtube Link

Members	Tasks
Brian King	
Houston Karnen	● VicRoads harvesters, including Elastic index + APIs

	● Report: System architecture, data sources, analysis & scenarios, conclusion
Jaden Ling	●
Klarissa Jutivannadevi	●
Lachlan Hugo	● BoM & EPA Harvesters incl. Elastic index setup. ● Fission API for BoM & EPA Harvesters ● Report ○ System architecture ○ Data sources ○ Fault tolerance & scalability